COMP8620

Advanced Topics in AI: Intelligent Robotics

Assignment 2 Report

Semester 2, 2026

Student	Rui Luo
UID	u8076655

The Australian National University

Generative AI Disclosure

OpenAI Codex was used to prepare the report template, review and revise the Part A answers, implement the Part B solver and tests, design and run the experiments, analyse their measured results, and prepare report text, pseudocode and figures. The supplied simulator's transition and reward methods were preserved; their use and the headless import changes are documented in Part B. Original-language prompts from these interactions, including supplied draft answers, are recorded in `AI_prompts.txt` in the submitted source archive.

Part A: Theory (30 points)

Question 1: Two-state MDP abstraction (10 points)

No.

Let $m = \lceil k/2 \rceil$, and define

$$\phi(s) = \begin{cases} L, & 1 \le s \le m, \\ R, & m+1 \le s \le k. \end{cases}$$

An exact MDP abstraction requires a transition function depending only on the abstract state and action while preserving the original motion probabilities. In particular, for every action a, transition consistency requires

$$\phi(s_1) = \phi(s_2) \implies \sum_{u:\, \phi(u)=R} T(s_1, a, u) = \sum_{u:\, \phi(u)=R} T(s_2, a, u).$$

For $k > 4$, consider $s_1 = m - 1$ and $s_2 = m$, both in L. Under $a = \text{moveRight}$,

$$m-1 \xrightarrow{0.7} m \in L, \qquad m-1 \xrightarrow{0.3} m-2 \in L,$$
$$m \xrightarrow{0.7} m+1 \in R, \qquad m \xrightarrow{0.3} m-1 \in L.$$

Hence,

$$P(R \mid s_1, a) = \sum_{u=m+1}^{k} T(m-1, a, u) = 0,$$
$$P(R \mid s_2, a) = \sum_{u=m+1}^{k} T(m, a, u) = T(m, a, m+1) = 0.7.$$

Since $0 \neq 0.7$, transition consistency fails: no single $T'(L, \text{moveRight}, R)$ can represent both states exactly.

Therefore, the proposed abstraction cannot preserve the original motion uncertainty and is not an exact MDP abstraction.

Question 2: Alpha-vector dominance (10 points)

Let $p = b(s_1)$. Since $b(s_1) + b(s_2) = 1$,

$$b = (p, 1-p), \qquad 0 \le p \le 1.$$

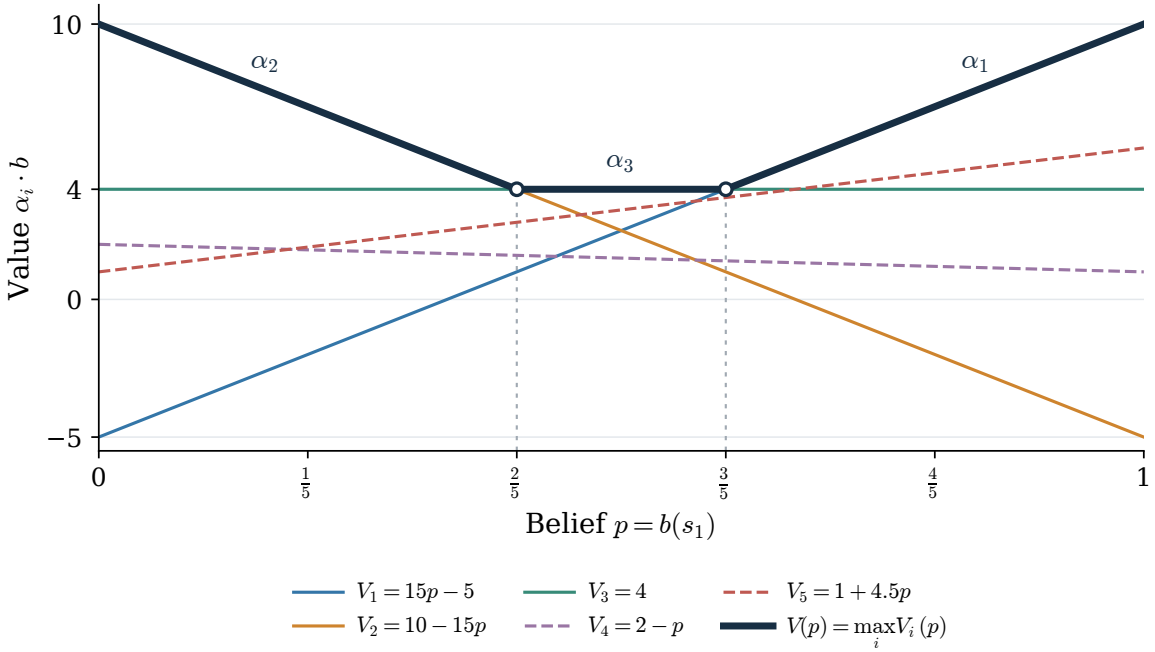
For each $\alpha_i \in \Gamma$, let $V_i(p) = \alpha_i \cdot b$. Then

$$\begin{aligned} V_1(p) &= 10p - 5(1-p) = 15p - 5, \\ V_2(p) &= -5p + 10(1-p) = 10 - 15p, \\ V_3(p) &= 4p + 4(1-p) = 4, \\ V_4(p) &= p + 2(1-p) = 2 - p, \\ V_5(p) &= 5.5p + (1-p) = 1 + 4.5p. \end{aligned}$$

(a) Maximal belief intervals (4 points)

The upper envelope is

$$V(p) = \max_{\alpha_i \in \Gamma} V_i(p).$$



The relevant intersections are

$$\begin{aligned} V_2(p) = V_3(p) &\iff 10 - 15p = 4 \iff p = \frac{2}{5}, \\ V_1(p) = V_3(p) &\iff 15p - 5 = 4 \iff p = \frac{3}{5}. \end{aligned}$$

The remaining vectors are never maximal, since

$$V_4(p) = 2 - p < 4 = V_3(p), \qquad 0 \le p \le 1,$$

and, using $V_1(p) - V_5(p) = 10.5p - 6$,

$$\begin{cases} V_5(p) \le 1 + 4.5 \times \frac{3}{5} = 3.7 < 4 = V_3(p), & 0 \le p \le \frac{3}{5}, \\ V_1(p) - V_5(p) \ge 10.5 \times \frac{3}{5} - 6 = 0.3 > 0, & \frac{3}{5} \le p \le 1. \end{cases}$$

Since V_2 decreases, V_1 increases, and $V_3 = 4$,

$$V(p) = \begin{cases} 10 - 15p, & 0 \le p < \frac{2}{5}, \\ 4, & \frac{2}{5} \le p \le \frac{3}{5}, \\ 15p - 5, & \frac{3}{5} < p \le 1. \end{cases}$$

Therefore, the maximal belief intervals are

$$\boxed{\begin{aligned} \alpha_1 &: \left[\frac{3}{5}, 1\right], \\ \alpha_2 &: \left[0, \frac{2}{5}\right], \\ \alpha_3 &: \left[\frac{2}{5}, \frac{3}{5}\right], \\ \alpha_4, \alpha_5 &: \varnothing. \end{aligned}}$$

At $p = \frac{2}{5}$, α_2 and α_3 tie; at $p = \frac{3}{5}$, α_1 and α_3 tie.

(b) Pointwise dominance (6 points)

An α-vector can be removed if, at every belief, another vector has strictly greater value:

$$\forall b \in B, \quad \exists \alpha' \in \Gamma : \alpha \cdot b < \alpha' \cdot b.$$

For α_4,

$$V_4(p) = 2 - p < 4 = V_3(p), \qquad \forall p \in [0, 1].$$

Thus, α_4 is strictly dominated by α_3.

For α_5, the dominating vector may depend on p. From part (a),

$$\begin{cases} V_5(p) < V_3(p), & 0 \le p \le \frac{3}{5}, \\ V_5(p) < V_1(p), & \frac{3}{5} < p \le 1. \end{cases}$$

These intervals cover $[0, 1]$, so α_5 also satisfies the removal condition.

The remaining vectors α_2, α_3, and α_1 are uniquely maximal at $p = 0$, $p = \frac{1}{2}$, and $p = 1$, respectively. Hence, none satisfies the removal condition.

Therefore,

$$\boxed{\Gamma_{\text{remove}} = \{\alpha_4, \alpha_5\}, \qquad \Gamma_{\text{keep}} = \{\alpha_1, \alpha_2, \alpha_3\}.}$$

Question 3: MDP versus POMDP for pedestrian avoidance (10 points)

Let c_t, v_t be the car's position and speed, and p_t^i the position of pedestrian i. Define

$$x_t = (c_t, v_t, p_t^1, \ldots, p_t^n).$$

Positions are observed accurately, and the car measures its own speed. It chooses an action a_t (accelerate, maintain speed, or brake) to reach G while avoiding collisions.

(a) Scenario suitable for an MDP

Suppose pedestrians follow known stochastic movement rules: the probabilities of crossing or continuing along the sidewalk depend only on the current observable configuration, with no persistent hidden intention. These memoryless dynamics give

$$P(x_{t+1} \mid x_{0:t}, a_{0:t}) = P(x_{t+1} \mid x_t, a_t).$$

Thus, x_t is Markov and fully observed, so an MDP with state $s_t = x_t$ is suitable despite the stochastic pedestrian motion.

(b) Scenario suitable for a POMDP

Now suppose each pedestrian has a persistent but unobservable destination g_i, such as crossing the road or continuing along the sidewalk. Let $\mathbf{g} = (g_1, \ldots, g_n)$, and assume motion is Markov conditional on these destinations. The complete state and observation are

$$s_t = (x_t, \mathbf{g}), \qquad o_t = x_t.$$

At the same observed configuration, two possible joint destinations can yield different motion distributions:

$$P(x_{t+1} \mid x_t, a_t, \mathbf{g}^{(1)}) \neq P(x_{t+1} \mid x_t, a_t, \mathbf{g}^{(2)}).$$

Different motion histories can produce the same x_t but different beliefs about $\mathbf{g}$. Hence, x_t alone need not be Markov. The car maintains

$$b_t(\mathbf{g}) = P(\mathbf{g} \mid x_{0:t}, a_{0:t-1})$$

and predicts motion using

$$P(x_{t+1} \mid x_t, b_t, a_t) = \sum_{\mathbf{g}} P(x_{t+1} \mid x_t, a_t, \mathbf{g})\, b_t(\mathbf{g}).$$

The belief is updated as new movements are observed. A POMDP is suitable because it uses both the observed configuration and the belief over hidden destinations to select driving actions.

Part B: Multi-drone planning under uncertainty (35 points)

Question 1: MDP formulation (3 points)

Let $\mathcal{G} = \{0, \dots, n_x - 1\} \times \{0, \dots, n_y - 1\} \times \{0, \dots, n_z - 1\}$, with obstacle cells $O \subset \mathcal{G}$. Drone i has a known goal g_i. A simulator state contains its cell p_i and reached flag r_i; include elapsed time t for the finite step limit:

$$s = ((p_i, r_i)_{i=1}^N, t), \qquad s_0 = ((p_i^0, \mathbf{1}\{p_i^0 = g_i\})_{i=1}^N, 0).$$

The positions and reached flags are observed. A reached drone remains at its goal and continues to occupy that cell. Success, collision and the step limit terminate the episode; the planner assigns zero continuation value after termination.

For altitude-changing flight, each intended move is

$$u_i \in \{-1, 0, 1\}^3 \setminus \{(0, 0, 0)\}, \qquad \mathcal{A} = \{0, \dots, 26^N - 1\}.$$

The supplied encoding is little-endian: $a = \sum_{i=1}^N a_i B^{i-1}$, where $B = 26$, or $B = 8$ for fixed altitude. There is no selectable hover action. Reached drones ignore their commands.

The known transition model samples each drone's raw target independently, conditional on the joint state and intended actions, then checks joint collisions. Writing q_i for the supplied per-drone target distribution and F for its target-to-state/termination mapping,

$$T(s, a, s') = \sum_{y : F(s,y) = s'} \prod_{i=1}^N q_i(y_i \mid p_i, r_i, a_i).$$

For an unreached drone, the intended direction has weight $d/(d + \alpha)$ and every other physical direction has weight $(1 - d/(d + \alpha))/(B - 1)$, where d is the supplied distance-field value. Out-of-grid directions receive zero weight and the remaining weights are renormalised. Obstacle targets are not removed: hitting one ends the mission. The code retains that drone's previous cell in the returned terminal state. Two drones in the same returned cell also cause termination. Cell swaps and diagonal segment crossings are allowed by the stated cell-occupancy model.

Use the provided transition reward, rather than an additional reward shaping term. For step cost c, collision penalty C and goal reward G,

$$R_{\text{sample}} = Nc + C(C_{\text{obs}} + C_{\text{veh}}) + G \sum_i \mathbf{1}\{r_i = 0,\ r_i' = 1\}.$$

Here C_{obs} counts obstacle hits, and C_{veh} counts all drones in shared-cell groups. The objective is

$$\max_\pi \ \mathbb{E}_\pi \left[\sum_{t=0}^{\tau - 1} \gamma^t R_t \right], \qquad \tau = \min\{T_{\text{success}}, T_{\text{collision}}, H\}.$$

Negative step costs favour fast completion; collision penalties discourage failure. This expected-return objective does not guarantee zero collision probability under stochastic motion.

Question 2: Approximate online solver (4 points)

I use the lectures' generative-model UCT/MCTS, with guided action widening for the exponentially large joint action set [1]. Unlike A1's RRT-Connect, it evaluates stochastic outcomes and replans after every observed transition. Full value/policy iteration enumerates too many joint states; exact RTDP backups also require action and successor sums. MCTS directly uses the supplied generative interface. Belief-space solvers are unnecessary for this fully observed problem, while model-free RL would require a separate training and generalisation study [2, 3].

The four components are specified as follows.

(1) **Selection.** Choose an unvisited admitted action first; otherwise maximise

$$Q(s,a) + 0.35\sqrt{\frac{\log(N(s)+1)}{N(s,a)}}.$$

Returns and heuristic values are divided by $\kappa = \max\{1, N(|G| + 2|C|)\}$ so the exploration constant has a comparable scale across drone counts.

(2) **Expansion.** Admit at most $\min\{|\mathcal{A}_{\text{eff}}(s)|, \lceil 1.5\sqrt{N(s)+1}\rceil\}$ actions. The first proposal is guided; later proposals use a uniform joint command with probability 0.15, otherwise a randomised guided command. Duplicate proposals are retried at most 12 times. Commands of already reached drones are canonically encoded as zero. Each distinct sampled successor has its own child; stochastic outcomes are never replaced by the most likely outcome.

(3) **Rollout.** At the first new successor, simulate up to eight steps: a uniform joint command with probability 0.10, otherwise the deterministic guided command. The tree depth is capped at eight. Terminal continuation is zero; other cutoffs use the heuristic below.

(4) **Backup.** With $z = R/\kappa + \gamma v$, update the sampled state–action edge by the incremental mean [2]:

$$N(s,a) \leftarrow N(s,a) + 1, \qquad Q(s,a) \leftarrow Q(s,a) + \frac{z - Q(s,a)}{N(s,a)}.$$

Increment $N(s)$ and repeat. Return the root action with most visits, breaking ties by Q; use the guided fallback if no simulation finishes.

Guidance and leaf estimate. For each goal, breadth-first search computes static free-cell distances $d_i(p)$ using the physical eight/26 neighbours. This deterministic relaxation ignores other drones and slips. Intended-action scores are

$$d_i(p_i + u_i) + 0.025\|p_i + u_i - g_i\|_1 + \frac{0.3}{\max\{1, d_{\text{clear}}(p_i + u_i)\}}.$$

Out-of-grid/obstacle targets receive cost 10^6; unreachable targets use distance cost 10^4. Goal-occupied and already reserved target cells receive an additional 10^5. Process active drones in index order for the fallback and in random order for stochastic proposals. The randomised proposal samples among the best four commands within 1.05 of the minimum, with weights $\exp[-3(\text{score} - \text{score}_{\min})]$. These preferences guide search; the simulator still determines all actual collisions and rewards.

With h steps remaining, let $D = \min\{h, \max_{i:r_i=0} d_i(p_i)\}$, using $D = h$ if an active goal is statically unreachable. The leaf estimate is

$$\widehat{V}(s,h) = Nc \sum_{j=0}^{D-1} \gamma^j + G \sum_{\substack{i:r_i=0 \\ 1 \le d_i(p_i) \le h}} \gamma^{d_i(p_i)-1}.$$

It is zero after success or when $h = 0$. This estimate approximates team completion and one-time goal rewards; it omits future collision risk and is not an exact value function. Maps are built once and their setup time is reported separately. Above 150,000 grid cells the implementation uses a Chebyshev-distance fallback and omits the clearance term to bound preprocessing memory.

Listing 1: Budgeted UCT with sampled outcomes

```
PROCEDURE PLAN(s, budget)
    deadline <- NOW + max(0, 0.97 * min(budget, 10) - 0.002)
    fallback <- GUIDED(s); root <- empty node
    h <- H - number of previously executed decisions
    WHILE before deadline AND trials < 2000 AND h > 0:
        IF all drones reached: BREAK
        SEARCH(root, s, h, 0)
    RETURN most-visited root action (tie: greatest Q), or fallback

PROCEDURE SEARCH(node, s, h, depth)
    IF h = 0 OR all drones reached: RETURN 0
    IF depth >= 8: RETURN ROLLOUT(s, h)
    admit a new proposed action if widening permits
    a <- unvisited action, otherwise UCB-maximising action
    (s_next, r, done) <- SIMULATE(s, a) // check deadline first
    IF done: v <- 0
    ELSE IF this outcome child is new:
        create child; v <- ROLLOUT(s_next, h - 1)
    ELSE:
        v <- SEARCH(child, s_next, h - 1, depth + 1)
    z <- r / kappa + gamma * v
    increment edge visits; update Q by incremental mean
    increment node visits; RETURN z

PROCEDURE ROLLOUT(s, h)
    simulate up to min(8, h) epsilon-greedy guided steps
    check deadline before each simulator call
    stop immediately on success or collision
    tail <- 0 if terminal, otherwise Vhat(s, remaining) / kappa
    RETURN discounted sampled rewards plus discounted tail
```

Interrupted incomplete tree trials are discarded. The remaining horizon is decremented in both tree search and rollout because the supplied `simulate` does not enforce the execution step limit.

Question 3: Implementation and demonstration (8 points)

The submission entry point is `run_u8076655.py`; `planner.py` implements B2. The supplied simulator is vendored at revision `ea3cbfe`, recorded with hashes in `UPSTREAM.json`. Only Vedo imports are deferred to visualisation methods; a subclass disables the plotting hooks for headless runs. AST comparison verifies that the constructor, reset, action encoding, dynamics, rewards and execution methods are unchanged. The original checkout is preserved.

The planner accesses only `get_config()`, `simulate()`, and the action-count interface, using its actual property form and also accepting a callable form [4]. Static heuristic maps are derived from the public configuration. It never inspects a future random outcome or changes an execution state during planning. Independent simulator instances use seeds s for execution and $1{,}000{,}003 + s$ for planning; the proposal RNG uses $2{,}000{,}003 + s$. Tutorial Week 8's sampling-quality concern is addressed with NumPy generators, while Week 9's incremental averaging is used in the backups [2].

```
# From the extracted submission's top-level folder:
python run_u8076655.py --config environments/demo_open_2d.yaml
python run_u8076655.py --config environments/demo_four_3d.yaml
# The marker's YAML requires no source edits:
python run_u8076655.py --config <scenario.yaml>
# Optional diagnostics / viewer:
python run_u8076655.py --config <scenario.yaml> --seed 3 \
  --planning-time 0.5 --output results/trajectory.json
python run_u8076655.py --config <scenario.yaml> --visualize
# Correctness tests, benchmarks and analysis:
python -m unittest discover -s tests -v
python scripts/run_all_experiments.py
python scripts/analyse_results.py
```

NumPy, SciPy and PyYAML are sufficient for headless execution; Matplotlib generates figures. Vedo/VTK is optional for the viewer. There is no training checkpoint or external data dependency. The default per-decision allowance is 9.5 seconds, with at most 2,000 trials. Python search is serial and the runner sets numerical-library thread limits to one.

The demo cases cover one fixed-altitude drone (8 actions) through four altitude-changing drones and five obstacle cells ($26^4 = 456{,}976$ joint actions). The latter is a stress case beyond the stated < 50-action expected test. Widening avoids constructing the full joint action list. The 25 unit tests pass. A separate exhaustive check of the free $2 \times 2 \times 1$ two-drone model covers 11 admissible nonterminal joint states, 704 state–action pairs and all 6,048 positive-probability outcomes. This is exhaustive only for that small model. The named demo cases complete in the following single seed-0 checks using the default 9.5-second allowance.

Task	Seed	Status	Steps	Max decision (s)
Open 2D	0	success	14	6.526
Four 3D, five obstacles	0	success	13	7.829

A forced 10-second call, with the trial cap raised to 100,000, returns in 9.699 seconds. These measured checks support the timing claim on this machine; they are not a hard real-time guarantee or evidence of success on every unseen layout.

Question 4: Experimental investigation (20 points)

Design and controls

The main study is a balanced $3 \times 3 \times 3$ factorial design with two drones. Starts $(0, 1, 1), (0, 3, 1)$, goals $(11, 1, 1), (11, 3, 1)$, $\alpha = 0.25$, $c = -1$, $C = -100$, $G = 100$, $\gamma = 0.98$, and $H = 60$ are fixed. Each of the 27 conditions uses seeds 0–19 and a 0.04-second planning allowance per decision: 540 episodes.

Factor	Levels	What the comparison measures
State size	Grid $(12, n_y, 3)$, $n_y = 5, 10, 20$	Expand the free region while retaining the task and obstacle coordinates.
Action size	6, 18, 26 intended directions per drone; $\lvert A\rvert = 36, 324, 676$	Axial, axial-plus-edge, or all neighbour commands, on the same physical model.
Clutter	0, 2, 5 nested obstacle cells	Introduce obstacles near the common start–goal routes.

For $F = 36n_y - |O|$ free cells, F^2 is a positional state-count upper bound, rather than an enumerated reachable-state count. The clock multiplier is constant across conditions. Action masks restrict intended commands only: each retained command calls the same original 26-direction transition model, including its slip outcomes. This preserves retained transition kernels; changing the simulator's altitude flag would not.

The state comparison tests whether global state count affects a focused online search or mainly its map setup. Action widening is expected to limit the cost of many joint commands, though added commands may improve recovery. Clutter can increase detours and collisions; because the supplied noise depends on obstacle distance, geometry and motion uncertainty change together. Expanding the grid also changes distant boundaries, and action masks change control capability. Thus these experiments estimate effects over the specified problem families, not isolated universal scaling laws.

Four representative tasks additionally test the full solver, uniform proposals/rollouts, zero leaf values, removal of widening, and guided control without search (20 seeds per cell; 400 episodes). A budget study uses 0.02, 0.1 and 0.5 seconds on three tasks (10 seeds; 90 episodes). Eighteen regression scenarios (20 seeds) and sixteen generated random layouts (10 seeds) cover 1–4 drones, 2D/3D movement, goals already occupied, narrow passages, boundaries, strong wind, infeasibility and timeouts. Together the recorded studies contain 1,550 executed episodes; they are not an exhaustive set of all possible environments.

Metrics and confidence intervals

Success requires all goals reached without collision within the step limit. To prevent early collision from appearing fast, define penalised completion length

$$L = \begin{cases} \tau, & \text{success}, \\ H, & \text{failure}. \end{cases}$$

Report L, discounted return, success counts, map/setup time, planning time, simulation calls and admitted root actions. Failures remain in the means. For episode-level continuous metrics, use the two-sided Student-t interval

$$\bar{x} \pm t_{0.975,n-1} \frac{s_x}{\sqrt{n}}.$$

These mean intervals are approximate for mixed success/failure distributions. Success proportions have Wilson 95% intervals in `summary.csv`. For factorial marginal means, first average the nine conditions within each seed block, then form a t interval across 20 blocks. High-minus-low factor effects and ablation effects use paired seed-block differences. This retains the dependence introduced by reusing seeds across conditions. Experiments ran serially on macOS 26.6.2 (`x86_64`), using Python 3.12.9, NumPy 1.26.4, SciPy 1.15.1, and PyYAML 6.0.2. No GPU, multiprocessing or threaded planning was used. Setup is timed separately from online decisions.

Measured results

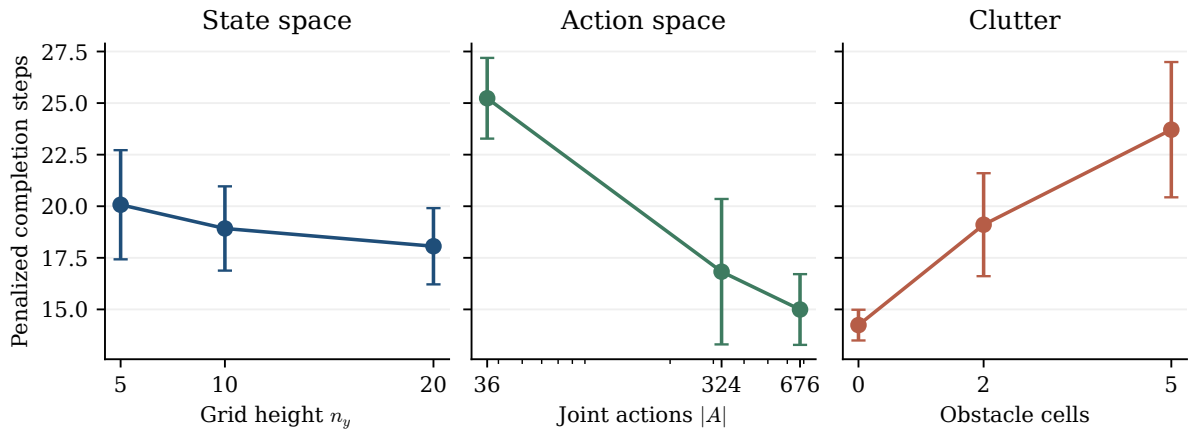


Figure 1: Factorial marginal means of penalised completion length. Error bars are 95% confidence intervals over 20 seed-block means.

Table 1: High-minus-low factor effects on L, averaged over the other factors within each seed. Positive values indicate degradation.

Factor	Mean change in L	95% interval
State size	-2.0	[-4.4, 0.3]
Action size	-10.2	[-12.5, -8.0]
Clutter	9.5	[6.4, 12.5]

Clutter is the clearest degrading factor in this design. Increasing obstacles from 0 to 5 raises L by 9.5 ± 3.0 steps; its 95% interval $[6.4, 12.5]$ excludes zero. Its effect exceeds the state-size effect by 11.5 steps (95% CI $[7.1, 15.9]$) and the action-size effect by 19.7 steps (95% CI $[15.9, 23.5]$). This is evidence for a ranking within these tested levels, not a claim that obstacle count always dominates every other source of difficulty. Obstacles affect both routes and the supplied distance-dependent noise.

State count alone does not explain online difficulty. Increasing n_y from 5 to 20 changes L by -2.0 ± 2.3 steps, with interval $[-4.4, 0.3]$. There is no clear degradation. Mean setup time increases from 18.1 to 63.2 ms because goal-distance maps cover more cells, but search does not enumerate the F^2 joint configurations. The wider workspace also offers more recovery space, so this comparison cannot isolate an abstract state-count effect from all geometric changes.

More available actions improve these tasks. Increasing the joint intended-action count from 36 to 676 changes L by -10.2 ± 2.2 steps, with interval $[-12.5, -8.0]$. Diagonal commands can combine forward progress with recovery from lateral/vertical slips, and the heuristic's deterministic relaxation is closer to the richer action set. These are plausible explanations, rather

than separately identified causes. The mean number of admitted root actions grows only from 5.8 to 6.4; widening prevents the coded action count from translating directly into full enumeration. The experiment therefore measures the net effect of additional control choices and search cost, not a pure branching-factor penalty.

Table 2: All 27 factorial conditions, $n = 20$ each. Continuous metrics show mean ± 95% Student-t CI half-width. Planning time is the episode's mean decision time, in milliseconds.

n_y	$\lvert A\rvert$	$\lvert O\rvert$	Success	L	Return	Plan (ms)
5	36	0	20/20	16.8 ± 1.3	121.5 ± 4.6	37.00 ± 0.02
5	36	2	17/20	27.9 ± 6.8	70.6 ± 38.5	37.07 ± 0.19
5	36	5	14/20	35.5 ± 8.0	32.8 ± 42.7	37.00 ± 0.03
5	324	0	20/20	12.2 ± 0.5	139.4 ± 1.8	37.17 ± 0.40
5	324	2	18/20	17.4 ± 6.8	109.0 ± 42.3	36.96 ± 0.02
5	324	5	15/20	24.9 ± 9.8	76.2 ± 49.4	37.28 ± 0.42
5	676	0	20/20	12.3 ± 0.7	139.2 ± 2.3	37.08 ± 0.16
5	676	2	19/20	15.5 ± 5.0	123.7 ± 25.0	37.05 ± 0.07
5	676	5	18/20	18.2 ± 6.7	110.7 ± 34.1	36.99 ± 0.04
10	36	0	20/20	16.8 ± 1.0	120.9 ± 3.8	37.00 ± 0.02
10	36	2	19/20	23.6 ± 4.5	93.4 ± 22.5	36.99 ± 0.01
10	36	5	15/20	35.5 ± 7.2	34.3 ± 43.4	37.01 ± 0.06
10	324	0	20/20	12.2 ± 0.6	139.3 ± 2.1	37.04 ± 0.12
10	324	2	19/20	15.5 ± 5.0	123.6 ± 25.0	37.02 ± 0.10
10	324	5	16/20	23.2 ± 8.9	89.8 ± 42.0	36.99 ± 0.02
10	676	0	20/20	12.2 ± 0.6	139.3 ± 2.2	36.99 ± 0.03
10	676	2	18/20	17.8 ± 6.8	112.5 ± 34.3	37.03 ± 0.09
10	676	5	20/20	13.4 ± 0.7	134.8 ± 2.4	37.06 ± 0.09
20	36	0	20/20	16.3 ± 1.2	122.0 ± 4.4	36.99 ± 0.02
20	36	2	19/20	23.8 ± 4.7	91.5 ± 22.4	37.04 ± 0.08
20	36	5	16/20	30.9 ± 7.1	55.5 ± 38.1	36.99 ± 0.02
20	324	0	20/20	12.2 ± 0.5	139.4 ± 1.8	37.05 ± 0.10
20	324	2	19/20	15.2 ± 5.0	124.2 ± 25.1	36.98 ± 0.02
20	324	5	18/20	18.6 ± 6.7	112.7 ± 28.7	36.98 ± 0.02
20	676	0	18/20	17.2 ± 6.9	105.0 ± 48.5	37.22 ± 0.44
20	676	2	19/20	15.2 ± 5.0	120.1 ± 35.6	36.98 ± 0.02
20	676	5	20/20	13.1 ± 0.8	136.0 ± 3.0	37.00 ± 0.02

Solver components

Table 3: Component comparisons, $n = 20$ per cell. Each entry gives success count followed by mean L±95% CI half-width; $H = 80$.

Variant	Open 2D	Four 3D	Swap 2D	Narrow
Full	20/20; 11.8 ± 0.5	17/20; 23.2 ± 11.5	20/20; 5.2 ± 0.3	16/20; 27.1 ± 12.7
Uniform actions	20/20; 28.9 ± 4.8	0/20; 80.0 ± 0.0	17/20; 35.6 ± 10.8	0/20; 80.0 ± 0.0
Zero leaf value	20/20; 12.1 ± 0.5	18/20; 19.8 ± 9.7	20/20; 5.3 ± 0.4	19/20; 17.5 ± 6.9
No widening	20/20; 11.8 ± 0.6	19/20; 16.3 ± 7.0	20/20; 5.2 ± 0.3	17/20; 23.3 ± 11.4
Guided only	20/20; 11.3 ± 0.3	18/20; 19.5 ± 9.7	20/20; 5.2 ± 0.3	17/20; 22.6 ± 11.6

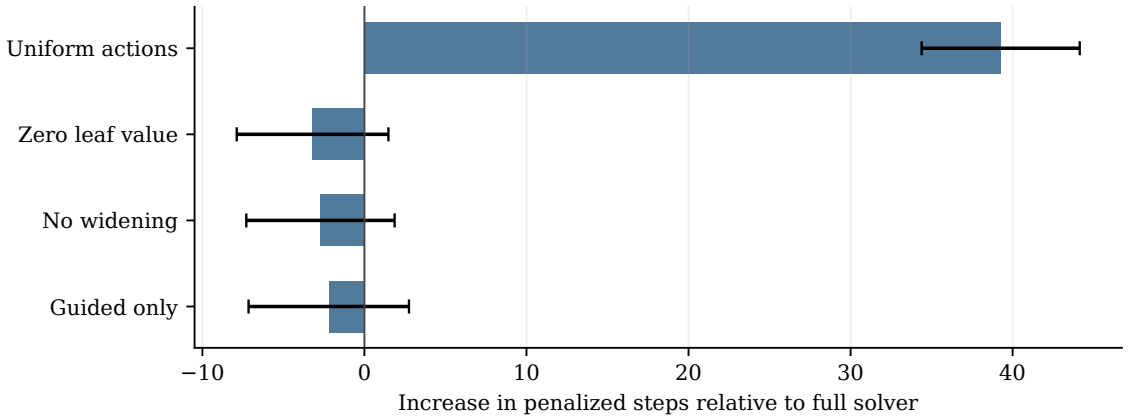


Figure 2: Paired changes in L relative to the full solver, averaged across four tasks within each seed. Error bars are 95% intervals across 20 seeds.

Replacing guided proposals and rollouts by uniform actions, while retaining the leaf estimate, increases L by 39.3 ± 4.9 steps across the four-task seed blocks. On Four 3D and Narrow, success falls to 0/20, compared with 17/20 and 16/20 for the full variant. Domain-informed action selection and rollout are therefore the strongest supported components in this study.

The other comparisons do not establish an improvement from the full configuration. Removing leaf values changes L by -3.2 ± 4.7; removing widening gives -2.7 ± 4.6; guided control without search gives -2.2 ± 4.9. All three intervals include zero. The negative point estimates favour simpler variants, but the data do not resolve their differences. Widening controls storage and exploration allocation; the short budget already limits growth. The leaf estimate ignores coupled occupancy and future slip risk, so it can also bias finite-budget decisions. These results support guidance, while leaving the benefit of these additional approximations unconfirmed at 40 ms. The full configuration was specified before this comparison and is not claimed to be the best variant in every task.

For the four MCTS stages, selection scans only admitted actions, while expansion stores sampled outcomes rather than a full joint transition table. As stochasticity spreads probability across more successors, repeated visits to a particular child can become scarce; rollout quality then matters more. Incremental mean backup uses constant extra storage per edge instead of storing all previous returns, but rare collision penalties can still make finite-sample values noisy. These describe computational mechanisms; UCB selection and the mean-backup rule were not separately ablated.

Planning budget

Table 4: Budget sensitivity, $n = 10$ per condition. L and return show mean ±95% CI half-width. Max time is the largest measured decision time.

Task	Budget (s)	Success	L	Return	Max time (s)
Four 3D	0.02	7/10	33.1 ± 23.2	148.1 ± 150.3	0.019
Four 3D	0.1	8/10	26.1 ± 20.3	190.0 ± 132.4	0.096
Four 3D	0.5	9/10	19.3 ± 15.3	239.0 ± 88.6	0.485
Open 2D	0.02	10/10	11.3 ± 0.5	71.0 ± 1.2	0.028
Open 2D	0.1	10/10	11.8 ± 0.7	69.8 ± 1.8	0.096
Open 2D	0.5	10/10	12.2 ± 0.9	68.9 ± 2.1	0.496
Swap 2D	0.02	10/10	5.4 ± 0.6	173.4 ± 2.2	0.021
Swap 2D	0.1	10/10	5.4 ± 0.6	173.4 ± 2.2	0.143
Swap 2D	0.5	10/10	5.4 ± 0.6	173.1 ± 2.8	0.484

For Four 3D, observed success rises from 7/10 at 0.02 s to 9/10 at 0.5 s, and mean L falls from 33.1 to 19.3. The intervals are broad with ten seeds, so this is a trend rather than a precise improvement estimate. Open 2D succeeds in every run at all three budgets; Swap 2D also succeeds throughout and has unchanged mean $L = 5.4$. Additional planning is consequently more relevant to the harder joint task, but does not uniformly shorten the observed paths.

The table reports actual decision times rather than assuming that a clock check enforces the nominal allowance exactly. Some short calls overrun their small allowance (the largest 0.1-second call is 0.143 s); simulator calls, Python cleanup and scheduling occur between cooperative checks. Every decision in these 1,550 episodes remains below 0.496 s. The separate default-budget and forced-deadline checks in B3 test the assignment's 10-second limit directly.

Reproducibility and limits

All scenarios, generators, raw CSVs, per-suite metadata, seed-0 trajectories, summary CSV/JSON files and figure scripts are included in the source archive. `course_inventory.json` records the local course-material inventory; `model_verification.json` records the upstream-core comparison. Wall-clock stopping means a seed alone need not reproduce an identical search tree on a different run or machine. A fixed trial cap under a generous time limit is used for the deterministic reproducibility test.

The tested noise range and finite scenario set do not establish performance on every unseen layout. Sparse stochastic outcome trees, heuristic distance errors, occupied goals and narrow passages remain important limitations. Nonzero slip probability can make collision risk unavoidable; the supplied finite penalty is not a hard safety constraint. More computation may reduce estimation error, but does not guarantee a better individual outcome or an optimal policy. Reported factor rankings apply only to the tested levels and fixed reward/budget settings. Conditions ran serially in fixed order on one machine; timing comparisons also remain subject to machine load and thermal drift.

A Prompt log and source inventory

The archive's top-level folder is `u8076655_A2_code`.
The file `AI_prompts.txt` records the original-language prompts.

File/folder	Purpose
`run_u8076655.py`	Demo entry point and closed-loop execution.
`planner.py`	Budgeted UCT, guidance, leaf values and diagnostics.
`environment.py`	Headless hooks, input checks and experimental action masks.
`multi_drone*.py`	Supplied simulator and configuration class.
`tests/`	Unit tests for model semantics and planning behavior.
`environments/`	All 61 YAML scenarios and their design manifest.
`results/`	Raw data, provenance, verification and seed-0 traces.
`scripts/`	Scenario generation, serial experiments, analysis and verification.
`report/`	LaTeX sources, measured tables and figures.

B Regression coverage

The 18 named scenarios each use 20 seeds and a 0.04-second allowance. Infeasible cases and the one-step horizon case are expected failures; stochastic collisions in feasible cases remain recorded failures. All 16 generated layouts and the full 27-condition design are retained in the manifest and raw data. These tests complement the 25 unit checks and the exhaustive small-model outcome check.

Table 5: Named regression scenarios. L shows mean ±95% CI half-width; its failure penalty is the scenario's own step limit.

Scenario	Success	Collisions	L	Max plan (s)
`demo_open_2d`	20/20	0	11.7 ± 0.5	0.0380
`demo_four_3d`	19/20	1	16.3 ± 7.0	0.0378
`already_at_goals`	20/20	0	0.0 ± 0.0	0.0000
`single_cell_goal`	20/20	0	0.0 ± 0.0	0.0000
`one_reached`	20/20	0	8.6 ± 0.5	0.0437
`swap_2d`	20/20	0	5.2 ± 0.3	0.0379
`boundary_reverse`	20/20	0	6.0 ± 0.5	0.0377
`tiny_2d`	20/20	0	1.1 ± 0.1	0.0375
`thin_1d`	20/20	0	9.8 ± 1.4	0.0374
`infeasible_altitude`	0/20	0	80.0 ± 0.0	0.0000
`infeasible_wall`	0/20	0	80.0 ± 0.0	0.0000
`infeasible_shared_goal`	0/20	0	80.0 ± 0.0	0.0000
`horizon_one`	0/20	0	1.0 ± 0.0	0.0489
`narrow_passage`	16/20	4	27.5 ± 12.7	0.0383
`altitude_detour`	17/20	3	20.6 ± 12.0	0.0478
`strong_wind`	14/20	6	39.5 ± 13.0	0.0376
`three_3d`	18/20	2	17.4 ± 10.0	0.0525
`four_2d`	16/20	4	24.3 ± 13.4	0.0383

References

[1] H. Kurniawati, *Week 5: MDP Solving; Week 6: MDP Solving (continued)*, COMP4620/8620 lecture materials, Australian National University, 2026.

[2] H. Kurniawati, *Tutorials Weeks 6, 8 and 9*, COMP4620/8620, Australian National University, 2026.

[3] H. Kurniawati, *Reinforcement Learning, Weeks 8–11*, COMP4620/8620 lecture materials, Australian National University, 2026.

[4] RDLLab, *MultiDroneUnc*, supplied code and README, `https://github.com/RDLLab/MultiDroneUnc`. Local revision `ea3cbfe`; full revision and source hashes are recorded in `UPSTREAM.json`.